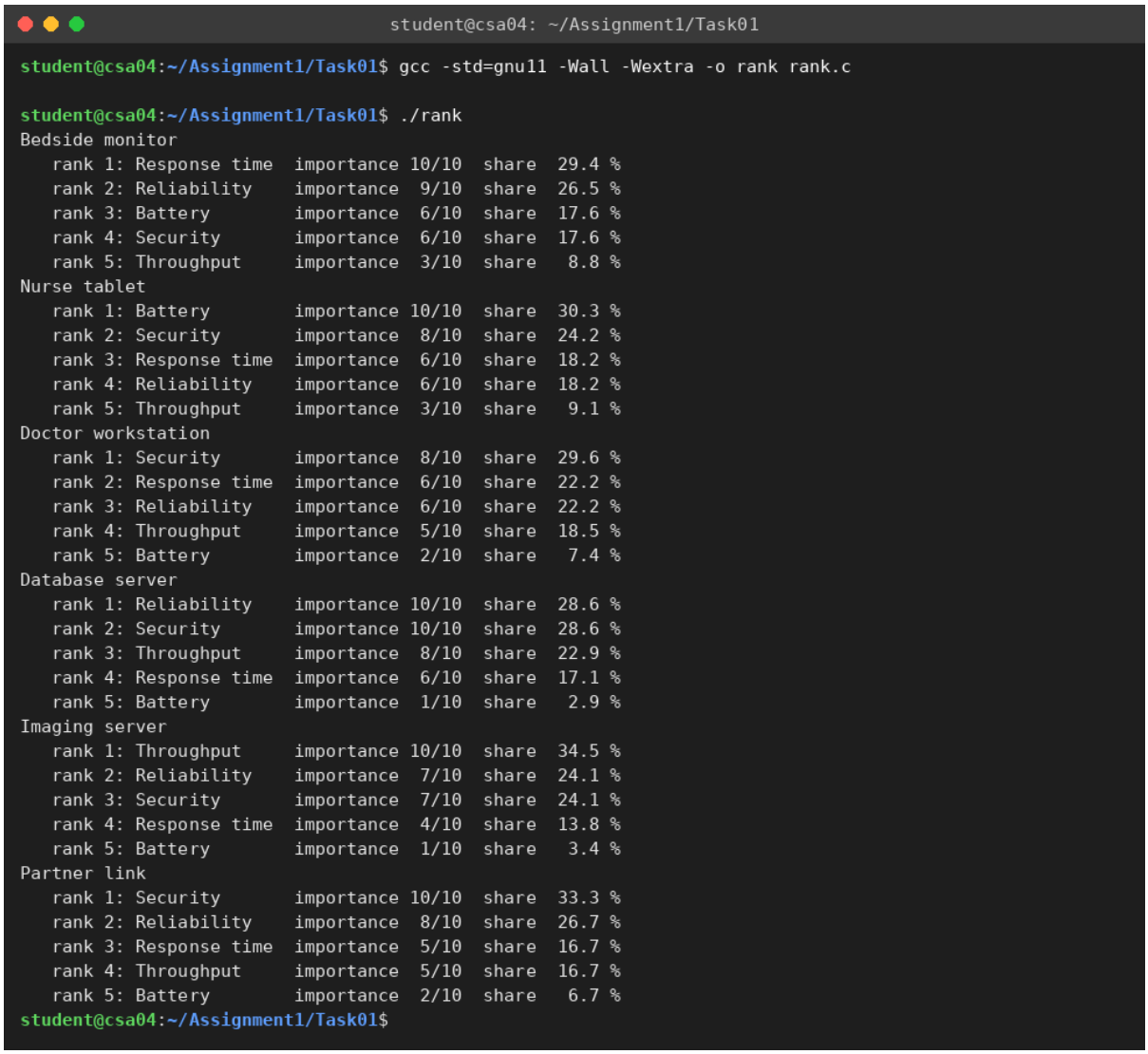
Q.No 1 – Evaluate the Requirements of Each Computing Environment

Figure 1.1: Output of rank.c



Source code: Task01/rank.c

rank.c

```c
/* rank.c - Task 1: weighted ranking of requirements for every environment.
 * Importance 1 (low) .. 10 (critical) is the evaluator's judgement; the program normalises and ranks.
 * Build: gcc -std=c11 -Wall -Wextra -o rank rank.c */
#include <stdio.h>

int main(void)
{
    const char *crit[5] = {"Response time", "Reliability", "Throughput", "Battery", "Security"};
    const char *env[6] = {"Bedside monitor", "Nurse tablet", "Doctor workstation",
                          "Database server", "Imaging server", "Partner link"};
    int w[6][5] = {
        {10, 9, 3, 6, 6},   /* bedside monitor   */
        { 6, 6, 3, 10, 8},  /* nurse tablet      */
        { 6, 6, 5, 2, 8},   /* doctor workstation*/
        { 6, 10, 8, 1, 10}, /* database server   */
        { 4, 7, 10, 1, 7},  /* imaging server    */
        { 5, 8, 5, 2, 10}   /* partner link      */
    };
    for (int e = 0; e < 6; e++) {
        int tot = 0, order[5] = {0, 1, 2, 3, 4};
        for (int i = 0; i < 5; i++) tot += w[e][i];
        for (int i = 1; i < 5; i++) {              /* stable insertion sort, highest importance first */
            int k = order[i], j = i - 1;
            while (j >= 0 && w[e][order[j]] < w[e][k]) { order[j + 1] = order[j]; j--; }
            order[j + 1] = k;
        }
        printf("%s\n", env[e]);
        for (int r = 0; r < 5; r++) {
            int i = order[r];
            printf("   rank %d: %-14s importance %2d/10  share %5.1f %%\n",
                   r + 1, crit[i], w[e][i], 100.0 * w[e][i] / tot);
        }
    }
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 1.2: Output of single_os.c

student@csa04: ~/Assignment1/Task01

```
student@csa04:~/Assignment1/Task01$ gcc -std=gnu11 -Wall -Wextra -o single_os single_os.c

student@csa04:~/Assignment1/Task01$ ./single_os
Worst-case alarm response time (budget = 10 ms)
  RTOS (hard real-time)      :     0.52 ms  -> deadline guaranteed
  General-purpose OS, loaded :   838.02 ms  -> unbounded in theory, 84x the deadline here
  General-purpose OS, idle   :     3.02 ms  -> usually fine, but no guarantee

Memory needed per device (MB, typical)        RTOS/special   general-purpose
  Bedside monitor (500)            1             2048
  Nurse tablet (120)            1024             4096
  Workstation (80)              4096             4096
  Fleet total (MB): special 451060   single OS 1843200   (x4.1)

Administration (person-hours per month, assumed 0.05 h per patch per device)
  one OS image, 704 devices, 2 patch cycles : 70 h
  4 OS families, 4 teams, 8 patch cycles      : 169 h  but each update is smaller and can be scoped
student@csa04:~/Assignment1/Task01$
```

Source code: Task01/single_os.c

single_os.c

```c
/* single_os.c - Task 1 alternative: one general-purpose OS everywhere.
 * Worst-case alarm delay, memory and administration estimate (typical / assumed values).
 * Build: gcc -std=c11 -Wall -Wextra -o single_os single_os.c */
#include <stdio.h>

int main(void)
{
    double isr_us = 20, rtos_wcet_ms = 0.5, quantum_ms = 100, page_fault_ms = 8, burst_ms = 30;
    int runq = 8;
    printf("Worst-case alarm response time (budget = 10 ms)\n");
    printf("  RTOS (hard real-time)      : %8.2f ms  -> deadline guaranteed\n", isr_us / 1000 + rtos_wcet_ms);
    double gp = isr_us / 1000 + runq * quantum_ms + page_fault_ms + burst_ms;
    printf("  General-purpose OS, loaded : %8.2f ms  -> unbounded in theory, %.0fx the deadline here\n", gp, gp / 10);
    printf("  General-purpose OS, idle   : %8.2f ms  -> usually fine, but no guarantee\n", isr_us / 1000 + 3);

    printf("\nMemory needed per device (MB, typical)        RTOS/special   general-purpose\n");
    const char *n[3] = {"Bedside monitor (500)", "Nurse tablet (120)", "Workstation (80)"};
    long a[3] = {1, 1024, 4096}, b[3] = {2048, 4096, 4096}, cnt[3] = {500, 120, 80};
    long ts = 0, tg = 0;
    for (int i = 0; i < 3; i++) {
        printf("  %-24s%10ld %16ld\n", n[i], a[i], b[i]);
        ts += cnt[i] * a[i]; tg += cnt[i] * b[i];
    }
    printf("  Fleet total (MB): special %ld   single OS %ld   (x%.1f)\n", ts, tg, (double)tg / ts);

    int dev = 500 + 120 + 80 + 4;
    printf("\nAdministration (person-hours per month, assumed 0.05 h per patch per device)\n");
    printf("  one OS image, %d devices, 2 patch cycles : %.0f h\n", dev, dev * 0.05 * 2);
    printf("  4 OS families, 4 teams, 8 patch cycles      : %.0f h  but each update is smaller and can be scoped\n", dev * 0.03 * 8);
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Q.No 2 – Judge the Type of Operating System for Each Environment

Figure 2.1: Output of availability.c

```
student@csa04: ~/Assignment1/Task02

student@csa04:~/Assignment1/Task02$ gcc -std=gnu11 -Wall -Wextra -o availability availability.c

student@csa04:~/Assignment1/Task02$ ./availability
Single large multiprocessor : availability 99.800 %  downtime   17.5 h/year (single point of failure)
2-node cluster, fail-over   : availability 99.9992 % downtime    4.3 min/year (fail-over in 30 s)

Cost (relative units): 1 big 32-core server = 10.0 ; 2 x 8-core servers + shared storage = 2*3.0 + 2.5 = 8.5

Throughput with p processors, parallel fraction f = 0.95 (Amdahl):  S(p) = 1/((1-f)+f/p)
   p= 1  speed-up  1.00   time for 1 h job:  60.0 min
   p= 2  speed-up  1.90   time for 1 h job:  31.5 min
   p= 4  speed-up  3.48   time for 1 h job:  17.3 min
   p= 8  speed-up  5.93   time for 1 h job:  10.1 min
   p=16  speed-up  9.14   time for 1 h job:   6.6 min
   p=32  speed-up 12.55   time for 1 h job:   4.8 min

Memory (space): big SMP shares ONE 512 GB pool ; cluster = 2 x 128 GB, data partitioned/replicated (needs 2x for repli
ca)
student@csa04:~/Assignment1/Task02$
```

Source code: Task02/availability.c

availability.c

```c
/* availability.c - Task 2 alternative: one large multiprocessor vs a 2-node cluster with fail-over.
 * Build: gcc -std=c11 -Wall -Wextra -o availability availability.c */
#include <stdio.h>

int main(void)
{
    const double H = 8760, mtbf = 2000, mttr = 4, failover_s = 30;
    double a1 = mtbf / (mtbf + mttr);
    printf("Single large multiprocessor : availability %.3f %%  downtime %6.1f h/year (single point of failure)\n",
           a1 * 100, (1 - a1) * H);
    double down = (H / mtbf) * failover_s / 3600 + H * (1 - a1) * (1 - a1);   /* fail-over gap + both nodes down */
    printf("2-node cluster, fail-over   : availability %.4f %% downtime %6.1f min/year (fail-over in %.0f s)\n",
           (1 - down / H) * 100, down * 60, failover_s);
    printf("\nCost (relative units): 1 big 32-core server = 10.0 ; 2 x 8-core servers + shared storage = 2*3.0 + 2.5 = 8.5\n");
    printf("\nThroughput with p processors, parallel fraction f = 0.95 (Amdahl):  S(p) = 1/((1-f)+f/p)\n");
    for (int p = 1; p <= 32; p *= 2) {
        double s = 1.0 / ((1 - 0.95) + 0.95 / p);
        printf("   p=%2d  speed-up %5.2f   time for 1 h job: %5.1f min\n", p, s, 60 / s);
    }
    printf("\nMemory (space): big SMP shares ONE 512 GB pool ; cluster = 2 x 128 GB, data partitioned/replicated (needs 2x for replica)\n");
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Q.No 3 – Evaluate Operating-System Structures

Figure 3.2: Compilation and run of ipc_bench.c

```
student@csa04: ~/Assignment1/Task03

student@csa04:~/Assignment1/Task03$ gcc -O2 -std=gnu11 -Wall -Wextra -o ipc_bench ipc_bench.c

student@csa04:~/Assignment1/Task03$ ./ipc_bench
ordinary function call      :       1.3 ns
system call (getppid)       :     113.6 ns   (90 x a function call)
message round trip (2 procs):    3256.7 ns   (29 x a system call)
student@csa04:~/Assignment1/Task03$
```

Source code: Task03/ipc_bench.c

ipc_bench.c

```c
/* ipc_bench.c - cost of (a) an ordinary function call, (b) a system call, (c) a message exchanged between two processes
 * (the cost model of a microkernel: a service request = send message + receive reply).
 * Build: gcc -O2 -std=gnu11 -Wall -Wextra ipc_bench.c -o ipc_bench */
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <time.h>
#include <sys/syscall.h>
#include <sys/wait.h>
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
__attribute__((noinline)) static long add1(long x){ __asm__ volatile(""); return x+1; }
int main(void){
    const long N=1000000; double t0; long s=0;
    t0=now(); for(long i=0;i<N;i++) s=add1(s); double fc=(now()-t0)/N;
    t0=now(); for(long i=0;i<N;i++) s+=syscall(SYS_getppid); double sc=(now()-t0)/N;
    int a[2],b[2]; if(pipe(a)||pipe(b)) return 1; char c='x'; const long M=100000;
    if(fork()==0){ for(long i=0;i<M;i++){ if(read(a[0],&c,1)!=1) _exit(1); if(write(b[1],&c,1)!=1) _exit(1);} _exit(0); }
    t0=now(); for(long i=0;i<M;i++){ if(write(a[1],&c,1)!=1) return 1; if(read(b[0],&c,1)!=1) return 1; }
    double mp=(now()-t0)/M; wait(NULL);
    printf("ordinary function call      : %9.1f ns\n",fc*1e9);
    printf("system call (getppid)       : %9.1f ns   (%.0f x a function call)\n",sc*1e9,sc/fc);
    printf("message round trip (2 procs): %9.1f ns   (%.0f x a system call)\n",mp*1e9,mp/sc);
    return (int)(s&0); }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Q.No 4 – Evaluate the Services an Operating System Must Provide

Source code: Task04/perm_demo.c (part 1 of 2)

perm_demo.c (1/2)

```c
/* perm_demo.c - Task 4: the OS protects one user's patient record from another user's program.
 * The owner creates a record with mode 0600; a child process then switches to user "nobody" (uid 65534)
 * and tries to open it.  Run as root (sudo).   Build: gcc -std=gnu11 -Wall -Wextra -o perm_demo perm_demo.c */
#define _GNU_SOURCE
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>

int main(void)
{
    const char *rec = "patient1042.rec";
    int fd = open(rec, O_WRONLY | O_CREAT | O_TRUNC, 0600);          /* owner read/write only */
    if (fd < 0) { perror("create"); return 1; }
    if (write(fd, "PATIENT 1042 DIAGNOSIS ...\n", 27) != 27) return 1;
    close(fd);
    struct stat st;
    stat(rec, &st);
    printf("record %s: owner uid=%d, mode=%o\n", rec, (int)st.st_uid, st.st_mode & 0777);

    fd = open(rec, O_RDONLY);
    printf("owner  (uid %d)    open() -> %s\n", (int)geteuid(), fd >= 0 ? "OK, access granted" : strerror(errno));
    if (fd >= 0) close(fd);

    fflush(stdout);                       /* empty the buffer before fork() (see Task 7) */
    pid_t p = fork();
    if (p == 0) {
        if (setgid(65534) != 0 || setuid(65534) != 0) { perror("setuid"); _exit(2); }
        fd = open(rec, O_RDONLY);
        printf("other  (uid %d) open() -> %s%s\n", (int)getuid(), fd >= 0 ? "OK" : "refused: ", fd >= 0 ? "" : strerror(errno));
        fflush(stdout);
        _exit(fd >= 0 ? 0 : 1);
    }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Source code: Task04/perm_demo.c (part 2 of 2)

perm_demo.c (2/2)

```c
    int status;
    waitpid(p, &status, 0);
    unlink(rec);
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 4.1: Compilation and run of perm_demo.c

student@csa04: ~/Assignment1/Task04

```
student@csa04:~/Assignment1/Task04$ gcc -std=gnu11 -Wall -Wextra -o perm_demo perm_demo.c

student@csa04:~/Assignment1/Task04$ ./perm_demo
record patient1042.rec: owner uid=0, mode=600
owner  (uid 0)    open() -> OK, access granted
other  (uid 65534) open() -> refused: Permission denied
student@csa04:~/Assignment1/Task04$
```

Figure 4.1b: Same check from the shell

```
student@csa04: ~/Assignment1/Task04

student@csa04:~/Assignment1/Task04$ id -un
root
student@csa04:~/Assignment1/Task04$ ls -l patient1042.rec
-rw------- 1 root root 27 Nov  5 05:14 patient1042.rec
student@csa04:~/Assignment1/Task04$ cat patient1042.rec
PATIENT 1042 DIAGNOSIS ...
student@csa04:~/Assignment1/Task04$ setpriv --reuid=65534 --regid=65534 --clear-groups id -un
nobody
student@csa04:~/Assignment1/Task04$ setpriv --reuid=65534 --regid=65534 --clear-groups cat patient1042.rec
cat: patient1042.rec: Permission denied
exit status = 1
student@csa04:~/Assignment1/Task04$
```

Source code: Task04/tablet.c

tablet.c

```c
/* tablet.c - Task 4 alternative: tablet with reduced handheld OS vs full desktop OS (typical/assumed figures).
 * Build: gcc -std=c11 -Wall -Wextra -o tablet tablet.c */
#include <stdio.h>

int main(void)
{
    const char *name[5] = {"Installed OS size (GB)", "RAM used at idle (GB)", "Background services running",
                           "Cold-start time (s)", "Average power during shift (W)"};
    double a[5] = {4.0, 0.8, 35, 12, 1.0}, b[5] = {30.0, 3.5, 180, 55, 3.2};
    int dec[5] = {1, 1, 0, 0, 1};
    printf("%-34s%12s%17s%8s\n", "quantity", "handheld OS", "full desktop OS", "ratio");
    for (int i = 0; i < 5; i++)
        printf("%-34s%12.*f%17.*f%7.1fx\n", name[i], dec[i], a[i], dec[i], b[i], b[i] / a[i]);
    double batt = 36.0;
    printf("\nBattery 36 Wh -> handheld OS lasts %.1f h ; desktop OS lasts %.1f h  (nurse shift = 12 h)\n", batt / 1.0, batt / 3.2);
    printf("Storage left for apps on a 64 GB tablet: handheld %.0f GB ; desktop %.0f GB\n", 64 - 4.0, 64 - 30.0);
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 4.2: Output of tablet.c

```
student@csa04: ~/Assignment1/Task04

student@csa04:~/Assignment1/Task04$ gcc -std=gnu11 -Wall -Wextra -o tablet tablet.c

student@csa04:~/Assignment1/Task04$ ./tablet
quantity                          handheld OS  full desktop OS   ratio
Installed OS size (GB)                    4.0             30.0    7.5x
RAM used at idle (GB)                     0.8              3.5    4.4x
Background services running                35              180    5.1x
Cold-start time (s)                        12               55    4.6x
Average power during shift (W)            1.0              3.2    3.2x

Battery 36 Wh -> handheld OS lasts 36.0 h ; desktop OS lasts 11.2 h  (nurse shift = 12 h)
Storage left for apps on a 64 GB tablet: handheld 60 GB ; desktop 34 GB
student@csa04:~/Assignment1/Task04$
```

Q.No 5 – Examine System Calls

Source code: Task05/sysdemo.c

sysdemo.c

```c
/* sysdemo.c - a program whose system calls we trace (Task 5) */
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <string.h>
#include <unistd.h>
int main(void)
{
    const char *rec = "PATIENT 1042 HR=78 SPO2=97\n";
    char buf[64];
    int fd = open("patient.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);   /* file management  */
    write(fd, rec, strlen(rec));                                         /* file management  */
    close(fd);
    fd = open("patient.txt", O_RDONLY);
    ssize_t n = read(fd, buf, sizeof buf);                               /* the read() we study */
    close(fd);
    write(STDOUT_FILENO, buf, (size_t)n);                                /* device management */
    getpid();                                                            /* information maintenance
    */
    unlink("patient.txt");
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 5.2: Compilation of sysdemo.c and system-call trace

student@csa04: ~/Assignment1/Task05

```
student@csa04:~/Assignment1/Task05$ gcc -std=c11 -Wall -Wextra -pthread -o sysdemo sysdemo.c

student@csa04:~/Assignment1/Task05$ ../tools/minitrace ./sysdemo 2>&1 | tail -10
openat(AT_FDCWD, "patient.txt", 0x241)                  = 3
write(3, "PATIENT 1042 HR=78 SPO2="..., 27)             = 27
close(3)                                                = 0
openat(AT_FDCWD, "patient.txt", 0x0)                    = 3
read(3, 0x7fffd7251ef0, 64)                             = 27
close(3)                                                = 0
PATIENT 1042 HR=78 SPO2=97
write(1, "PATIENT 1042 HR=78 SPO2="..., 27)             = 27
getpid(...)                                             = 16284
unlink("patient.txt")                                   = 0
student@csa04:~/Assignment1/Task05$
```

Source code: Task05/raw_read.c

raw_read.c

```c
/* raw_read.c - read a 1 MB file with read() in 100-byte pieces (every piece = one system call) */
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <unistd.h>
int main(void)
{
    char b[100];
    long t = 0;
    ssize_t n;
    int fd = open("big.dat", O_RDONLY);
    while ((n = read(fd, b, sizeof b)) > 0)
        t += n;
    close(fd);
    return t == 1048576 ? 0 : 1;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

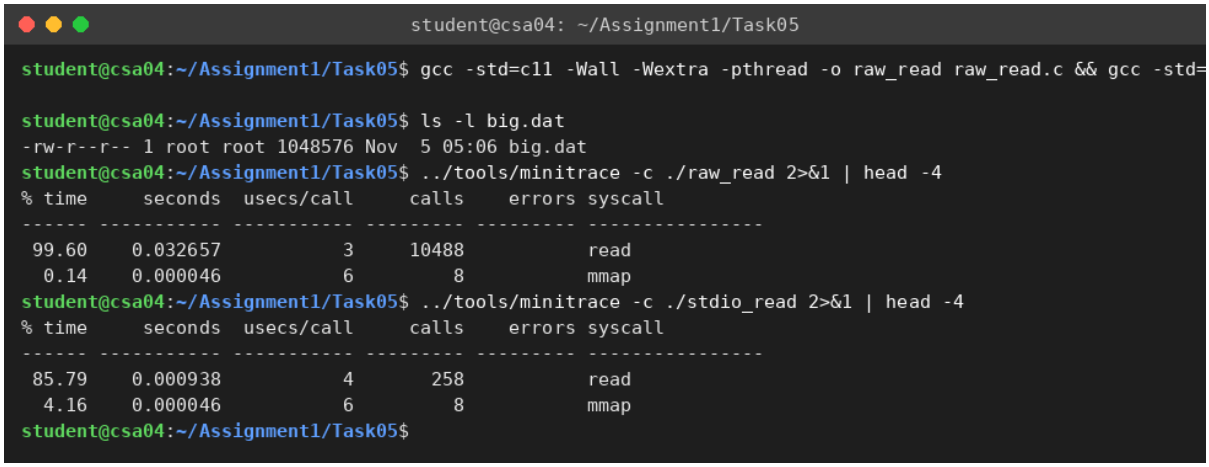
Source code: Task05/stdio_read.c

stdio_read.c

```c
/* stdio_read.c - read the same file with fread() in 100-byte pieces (library buffers 4096 bytes) */
#include <stdio.h>
int main(void)
{
    char b[100];
    long t = 0;
    size_t n;
    FILE *f = fopen("big.dat", "rb");
    while ((n = fread(b, 1, sizeof b, f)) > 0)
        t += n;
    fclose(f);
    return t == 1048576 ? 0 : 1;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 5.3: read() calls, raw read() vs fread()



Q.No 6 – Evaluate the Cost of System Calls

Source code: Task06/copy.c (part 1 of 2)

copy.c (1/2)

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
/* Copy src to dst with the read() and write() system calls, using a buffer of
 bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
 char buf[8192];
 int in = open(src, O_RDONLY);
 if (in < 0) return -1;
 int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
 if (out < 0) { close(in); return -1; }
 long calls = 0;
 ssize_t n;
 // TODO 1 (completed): loop with read(); count every read call; stop at EOF (0) or error (-1);
 // write exactly the n bytes that were read
 if (bufsize > sizeof buf) bufsize = sizeof buf;   /* boundary guard: never overrun buf */
 while (1) {
     n = read(in, buf, bufsize);
     calls++;                                /* count every read() call, including the last one */
     if (n <= 0) break;                      /* 0 = end of file, -1 = error */
     ssize_t done = 0;
     while (done < n) {                      /* write() may be partial: repeat until all n bytes writ
    ten */
         ssize_t w = write(out, buf + done, (size_t)(n - done));
         if (w < 0) { calls = -1; goto finish; }
         done += w;
     }
 }
 if (n < 0) calls = -1;
finish:
 close(in);
 close(out);
 return calls;
}
int main(void)
{
 FILE *f = fopen("records.dat", "w"); /* a 10,000-byte test file */
 for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Source code: Task06/copy.c (part 2 of 2)

copy.c (2/2)

```c
 fclose(f);
 size_t sizes[] = {1, 512, 4096};
 for (int i = 0; i < 3; i++)
 printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
 copy_file("records.dat", "copy.dat", sizes[i]));
 unlink("records.dat");
 unlink("copy.dat");
 return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 6.1: Compilation and execution of copy.c

student@csa04: ~/Assignment1/Task06

```
student@csa04:~/Assignment1/Task06$ gcc -std=c11 -Wall -Wextra -pthread -o copy copy.c

student@csa04:~/Assignment1/Task06$ ./copy
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
student@csa04:~/Assignment1/Task06$
```

Figure 6.2: Trace summary of copy.c

```
student@csa04: ~/Assignment1/Task06

student@csa04:~/Assignment1/Task06$ ../tools/minitrace -c ./copy 2>&1 | head -8
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 51.76    0.034462           3     10027           write
 46.31    0.030837           3     10027           read
  1.13    0.000752          84         9           openat
student@csa04:~/Assignment1/Task06$
```

Source code: Task06/bench_copy.c (part 1 of 2)

bench_copy.c (1/2)

```c
/* bench_copy.c - measure read()/write() copy cost for different buffer sizes,
 * compare with stdio (fread/fwrite) and mmap.   Build: gcc -O2 -std=gnu11 -Wall -Wextra bench_copy.c -o bench_copy */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/resource.h>

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
static void io_counts(long *r, long *w){            /* kernel counters: number of read()/write() calls so far */
    FILE *f=fopen("/proc/self/io","r"); char k[64]; long v; *r=*w=0;
    while(f && fscanf(f,"%63s %ld",k,&v)==2){ if(!strcmp(k,"syscr:")) *r=v; if(!strcmp(k,"syscw:")) *w=v; }
    if(f) fclose(f); }
static void cpu(double *u,double *s){ struct rusage ru; getrusage(RUSAGE_SELF,&ru);
    *u=ru.ru_utime.tv_sec+ru.ru_utime.tv_usec/1e6; *s=ru.ru_stime.tv_sec+ru.ru_stime.tv_usec/1e6; }

static void rw_copy(const char *a,const char *b,size_t bs){
    char *buf=malloc(bs); int in=open(a,O_RDONLY), out=open(b,O_WRONLY|O_CREAT|O_TRUNC,0644); ssize_t n;
    while((n=read(in,buf,bs))>0){ if(write(out,buf,n)!=n){perror("write");break;} }
    close(in); close(out); free(buf); }
static void stdio_copy(const char *a,const char *b){
    FILE *in=fopen(a,"r"), *out=fopen(b,"w"); int c;
    while((c=fgetc(in))!=EOF) fputc(c,out);          /* byte at a time, but buffered by the library */
    fclose(in); fclose(out); }
static void mmap_copy(const char *a,const char *b){
    int in=open(a,O_RDONLY), out=open(b,O_RDWR|O_CREAT|O_TRUNC,0644); struct stat st; fstat(in,&st);
    if(ftruncate(out,st.st_size)!=0) perror("ftruncate");
    char *s=mmap(NULL,st.st_size,PROT_READ,MAP_PRIVATE,in,0), *d=mmap(NULL,st.st_size,PROT_READ|PROT_WRITE,MAP_SHARED,out,0);
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Source code: Task06/bench_copy.c (part 2 of 2)

bench_copy.c (2/2)

```
    memcpy(d,s,st.st_size); munmap(s,st.st_size); munmap(d,st.st_size); close(in); close(out); }

int main(int argc,char **argv){
    size_t N=(argc>1)?strtoul(argv[1],0,10):8u*1024*1024;   /* file size in bytes */
    const char *src="bench_src.dat",*dst="bench_dst.dat";
    FILE *f=fopen(src,"w"); char blk[4096]; for(int i=0;i<4096;i++) blk[i]='A'+i%26;
    for(size_t i=0;i<N/4096;i++){ fwrite(blk,1,4096,f); }
    fclose(f);
    printf("File size n = %zu bytes (%.1f MB)\n\n",N,N/1048576.0);
    printf("%-22s %10s %10s %8s %8s %8s\n","method","read()","write()","real(s)","user(s)","sys(s)")
    ;
    size_t sizes[]={1,16,64,512,4096,65536,1048576};
    for(int i=0;i<7;i++){
        long r0,w0,r1,w1; double u0,s0,u1,s1; io_counts(&r0,&w0); cpu(&u0,&s0); double t0=now();
        rw_copy(src,dst,sizes[i]);
        double t1=now(); cpu(&u1,&s1); io_counts(&r1,&w1);
        char name[40]; snprintf(name,sizeof name,"read/write b=%zu",sizes[i]);
        printf("%-22s %10ld %10ld %8.3f %8.3f %8.3f\n",name,r1-r0,w1-w0,t1-t0,u1-u0,s1-s0); }
    { long r0,w0,r1,w1; double u0,s0,u1,s1; io_counts(&r0,&w0); cpu(&u0,&s0); double t0=now();
      stdio_copy(src,dst); double t1=now(); cpu(&u1,&s1); io_counts(&r1,&w1);
      printf("%-22s %10ld %10ld %8.3f %8.3f %8.3f\n","fgetc/fputc (stdio)",r1-r0,w1-w0,t1-t0,u1-u0,s
    1-s0); }
    { long r0,w0,r1,w1; double u0,s0,u1,s1; io_counts(&r0,&w0); cpu(&u0,&s0); double t0=now();
      mmap_copy(src,dst); double t1=now(); cpu(&u1,&s1); io_counts(&r1,&w1);
      printf("%-22s %10ld %10ld %8.3f %8.3f %8.3f\n","mmap + memcpy",r1-r0,w1-w0,t1-t0,u1-u0,s1-s0);
     }
    unlink(src); unlink(dst); return 0; }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 6.3: Measured system calls and times for a 4 MB copy

student@csa04: ~/Assignment1/Task06

```
student@csa04:~/Assignment1/Task06$ gcc -O2 -std=gnu11 -Wall -Wextra -o bench_copy bench_copy.c

student@csa04:~/Assignment1/Task06$ bash -c "time ./bench_copy 4194304"
File size n = 4194304 bytes (4.0 MB)

method                    read()    write()  real(s)  user(s)   sys(s)
read/write b=1           4194307    4194304    2.434    0.406    1.959
read/write b=16           262147     262144    0.140    0.020    0.119
read/write b=64            65539      65536    0.056    0.000    0.038
read/write b=512            8195       8192    0.017    0.000    0.007
read/write b=4096           1027       1024    0.008    0.000    0.003
read/write b=65536            67         64    0.007    0.000    0.002
read/write b=1048576           7          4    0.005    0.000    0.002
fgetc/fputc (stdio)         1027       1024    0.028    0.022    0.004
mmap + memcpy                  2          0    0.006    0.000    0.002

real  0m2.707s
user  0m0.449s
sys  0m2.142s
student@csa04:~/Assignment1/Task06$
```

Q.No 7 – Evaluate Process Control System Calls

Source code: Task07/shell.c

shell.c

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
 printf("shell: running job 1\n");
 fflush(stdout); /* empty the buffer before fork() */
 // TODO 1 (completed): fork(); child execs expr, parent waits and prints the exit status
 pid_t pid;
 int status;
 pid = fork();
 if (pid < 0) { perror("fork"); return 1; }
 if (pid == 0) {
     execlp("expr", "expr", "6", "*", "7", (char *)NULL);
     exit(127);                              /* reached only if exec failed */
 }
 waitpid(pid, &status, 0);
 printf("job 1 exit status %d\n", WEXITSTATUS(status));

 printf("shell: running job 2\n");
 fflush(stdout);
 pid = fork();
 if (pid == 0) _exit(3); /* a child that fails with code 3 */
 waitpid(pid, &status, 0);
 printf("job 2 exit status %d\n", WEXITSTATUS(status));
 return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 7.2: Compilation and execution of shell.c

```
student@csa04: ~/Assignment1/Task07

student@csa04:~/Assignment1/Task07$ gcc -std=c11 -Wall -Wextra -pthread -o shell shell.c

student@csa04:~/Assignment1/Task07$ ./shell
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
student@csa04:~/Assignment1/Task07$
```

Figure 7.3: fork (clone), waitpid (wait4) and write calls

```
student@csa04: ~/Assignment1/Task07

student@csa04:~/Assignment1/Task07$ ../tools/minitrace ./shell 2>&1 | grep -E "clone|wait4|write|exit_group
write(1, "shell: running job 1\n", 21)                  = 21
clone(...)                                              = 16344
wait4(...)                                              = 16344
write(1, "job 1 exit status 0\nshel"..., 41)            = 41
clone(...)                                              = 16345
wait4(...)                                              = 16345
write(1, "job 2 exit status 3\n", 20)                   = 20
student@csa04:~/Assignment1/Task07$
```

Source code: Task07/shell_noflush.c

shell_noflush.c

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
 printf("shell: running job 1\n");
 fflush(stdout); /* empty the buffer before fork() */
 // TODO 1 (completed): fork(); child execs expr, parent waits and prints the exit status
 pid_t pid;
 int status;
 pid = fork();
 if (pid < 0) { perror("fork"); return 1; }
 if (pid == 0) {
     execlp("expr", "expr", "6", "*", "7", (char *)NULL);
     exit(127);                              /* reached only if exec failed */
 }
 waitpid(pid, &status, 0);
 printf("job 1 exit status %d\n", WEXITSTATUS(status));

 printf("shell: running job 2\n");
 pid = fork();
 if (pid == 0) exit(3); /* a child that fails with code 3 */
 waitpid(pid, &status, 0);
 printf("job 2 exit status %d\n", WEXITSTATUS(status));
 return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 7.4: diff and redirected output

student@csa04: ~/Assignment1/Task07

```
student@csa04:~/Assignment1/Task07$ gcc -std=c11 -Wall -Wextra -pthread -o shell_noflush shell_noflush.c

student@csa04:~/Assignment1/Task07$ diff shell.c shell_noflush.c
23d22
<  fflush(stdout);
25c24
<  if (pid == 0) _exit(3); /* a child that fails with code 3 */
---
>  if (pid == 0) exit(3); /* a child that fails with code 3 */
student@csa04:~/Assignment1/Task07$ ./shell_noflush > out.txt

student@csa04:~/Assignment1/Task07$ cat out.txt
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
student@csa04:~/Assignment1/Task07$
```

Source code: Task07/spawn_bench.c

spawn_bench.c

```c
/* spawn_bench.c - time to start 1,000 commands: fork+exec vs vfork+exec vs posix_spawn,
 * with a small and a large parent (to show the cost of copying page tables).  (Task 7 alternative / Task 10) */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <spawn.h>
#include <time.h>
#include <sys/wait.h>
extern char **environ;
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
static double run(int mode,int n){
    char *argv[]={"/bin/true",NULL}; double t0=now();
    for(volatile int i=0;i<n;i++){
        pid_t p;
        if(mode==0){ p=fork(); if(p==0){ execv("/bin/true",argv); _exit(127);} }
        else if(mode==1){ p=vfork(); if(p==0){ execv("/bin/true",argv); _exit(127);} }
        else { if(posix_spawn(&p,"/bin/true",NULL,NULL,argv,environ)!=0){perror("spawn");exit(1);} }
        waitpid(p,NULL,0); }
    return now()-t0; }
int main(void){
    const int N=1000; const char *nm[]={"fork + exec","vfork + exec","posix_spawn"};
    for(int pass=0;pass<2;pass++){
        size_t mb = pass? 256:0; char *big=NULL;
        if(mb){ big=malloc(mb<<20); memset(big,1,mb<<20); }
        printf("Parent memory touched: %3zu MB\n",mb);
        for(int m=0;m<3;m++){ double t=run(m,N); printf("  %-14s %d commands: %7.3f s  (%6.1f us per command)\n",nm[m],N,t,t/N*1e6); }
        free(big); }
    return 0; }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 7.5: Time to start 1,000 commands

student@csa04: ~/Assignment1/Task07

```
student@csa04:~/Assignment1/Task07$ gcc -O2 -std=gnu11 -Wall -Wextra -o spawn_bench spawn_bench.c

student@csa04:~/Assignment1/Task07$ ./spawn_bench
Parent memory touched:   0 MB
  fork + exec    1000 commands:   1.127 s  (1127.4 us per command)
  vfork + exec   1000 commands:   1.024 s  (1024.1 us per command)
  posix_spawn    1000 commands:   1.026 s  (1025.9 us per command)
Parent memory touched: 256 MB
  fork + exec    1000 commands:   4.081 s  (4080.9 us per command)
  vfork + exec   1000 commands:   0.998 s  ( 998.0 us per command)
  posix_spawn    1000 commands:   1.056 s  (1056.0 us per command)
student@csa04:~/Assignment1/Task07$
```

Q.No 8 – Evaluate CPU States and Protection

Source code: Task08/poll_vs_interrupt.c

poll_vs_interrupt.c

```c
/* poll_vs_interrupt.c - wait 2 seconds for an "abnormal reading": (a) polling loop  (b) sleep until
     an interrupt-like signal.
 * Build: gcc -O2 -std=gnu11 -Wall -Wextra poll_vs_interrupt.c -o poll_vs_interrupt */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <time.h>
#include <sys/resource.h>
static volatile sig_atomic_t alarm_flag = 0;
static void on_alarm(int s){ (void)s; alarm_flag = 1; }
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_
    nsec/1e9; }
static double cpu(void){ struct rusage r; getrusage(RUSAGE_SELF,&r); return r.ru_utime.tv_sec+r.ru_s
    time.tv_sec+(r.ru_utime.tv_usec+r.ru_stime.tv_usec)/1e6; }
int main(void){
    struct sigaction sa={0}; sa.sa_handler=on_alarm; sigaction(SIGALRM,&sa,NULL);
    /* (a) polling: the sensor line is "abnormal" after 2 s; the CPU keeps testing the flag */
    alarm_flag=0; alarm(2); double c0=cpu(), t0=now(); long checks=0;
    while(!alarm_flag) checks++;
    printf("POLLING   : detected after %.3f s, CPU used %.3f s, %ld checks\n",now()-t0,cpu()-c0,chec
    ks);
    /* (b) interrupt-driven: the CPU sleeps (pause) and the signal wakes it */
    alarm_flag=0; alarm(2); c0=cpu(); t0=now();
    while(!alarm_flag) pause();
    printf("INTERRUPT : detected after %.3f s, CPU used %.3f s\n",now()-t0,cpu()-c0);
    return 0; }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 8.2: Polling vs interrupt-style waiting

student@csa04: ~/Assignment1/Task08

```
student@csa04:~/Assignment1/Task08$ gcc -O2 -std=gnu11 -Wall -Wextra -o poll_vs_interrupt poll_vs_interrupt

student@csa04:~/Assignment1/Task08$ bash -c "time ./poll_vs_interrupt"
POLLING   : detected after 2.000 s, CPU used 1.965 s, 5772462485 checks
INTERRUPT : detected after 2.000 s, CPU used 0.000 s

real  0m4.002s
user  0m1.967s
sys  0m0.000s
student@csa04:~/Assignment1/Task08$
```

Source code: Task08/latency.c

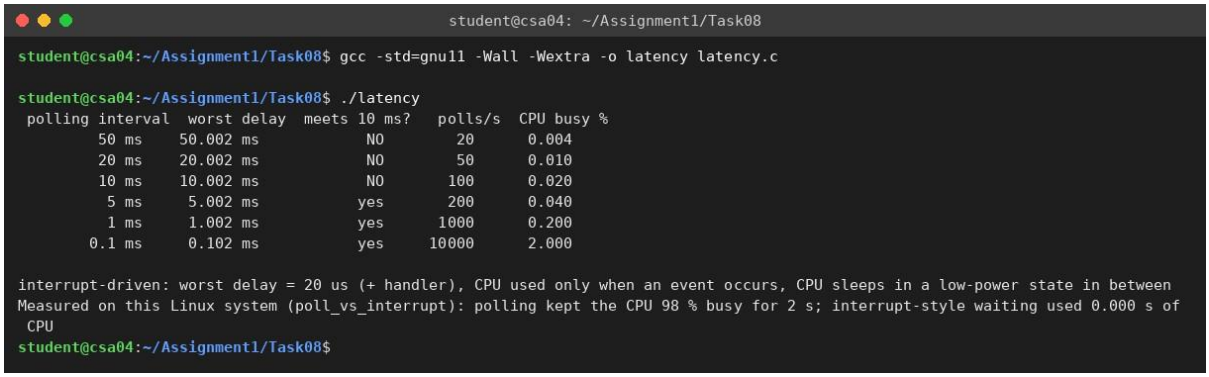
latency.c

```c
/* latency.c - Task 8 alternative: polling vs interrupt - worst-case delay and CPU load per monitor.
 * Build: gcc -std=c11 -Wall -Wextra -o latency latency.c */
#include <stdio.h>

int main(void)
{
    double cost_poll_us = 2.0, isr_latency_us = 20;
    double T[6] = {50, 20, 10, 5, 1, 0.1};
    printf("%17s %12s %13s %9s %11s\n", "polling interval", "worst delay", "meets 10 ms?", "polls/s"
    , "CPU busy %");
    for (int i = 0; i < 6; i++) {
        double worst = T[i] + cost_poll_us / 1000;
        printf("%11g ms %9.3f ms %13s %9.0f %10.3f\n", T[i], worst, worst < 9 ? "yes" : "NO",
               1000 / T[i], 100 * cost_poll_us * 1e-6 * 1000 / T[i]);
    }
    printf("\ninterrupt-driven: worst delay = %.0f us (+ handler), CPU used only when an event occur
    s, CPU sleeps in a low-power state in between\n", isr_latency_us);
    printf("Measured on this Linux system (poll_vs_interrupt): polling kept the CPU 98 %% busy for 2
     s; interrupt-style waiting used 0.000 s of CPU\n");
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 8.3: Worst-case detection delay for polling intervals



Q.No 9 – Evaluate the Memory Hierarchy and I/O Systems

Source code: Task09/memlat.c

memlat.c

```c
/* memlat.c - measure average access time of random reads in working sets of different size (Task 9: memory hierarchy).
 * Build: gcc -O2 -std=gnu11 -Wall -Wextra memlat.c -o memlat */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
int main(void){
    size_t kb[]={8,32,256,1024,8192,65536,262144};
    printf("%12s %14s\n","working set","ns per access");
    for(int s=0;s<7;s++){
        size_t n=kb[s]*1024/sizeof(size_t); size_t *a=malloc(n*sizeof *a);
        for(size_t i=0;i<n;i++) a[i]=i;
        srand(1); for(size_t i=n-1;i>0;i--){ size_t j=((size_t)rand()*65537u+rand())%(i); size_t t=a[i]; a[i]=a[j]; a[j]=t; } /* Sattolo: one random cycle */
        size_t p=0, steps=20000000; double t0=now();
        for(size_t i=0;i<steps;i++) p=a[p];                          /* every load depends on the previous one */
        double t=now()-t0;
        if(kb[s]>=1024) printf("%9zu MB %14.2f   (%zu)\n",kb[s]/1024,t/steps*1e9,p%2); else printf("%9zu KB %14.2f   (%zu)\n",kb[s],t/steps*1e9,p%2);
        free(a); }
    return 0; }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 9.2: Time per random access vs working-set size

```
student@csa04: ~/Assignment1/Task09
student@csa04:~/Assignment1/Task09$ gcc -O2 -std=gnu11 -Wall -Wextra -o memlat memlat.c

student@csa04:~/Assignment1/Task09$ ./memlat
 working set  ns per access
        8 KB          1.67   (1)
       32 KB          1.70   (0)
      256 KB          4.39   (0)
        1 MB         10.06   (1)
        8 MB        110.88   (0)
       64 MB        149.41   (1)
      256 MB        168.26   (0)
student@csa04:~/Assignment1/Task09$
```

Source code: Task09/sigcost.c

sigcost.c

```c
/* sigcost.c - measure the cost of one asynchronous event delivered to a handler (software analogue of an interrupt).
 * Used in Task 9 to estimate the CPU time of "one interrupt per word".  Build: gcc -O2 -std=gnu11 -Wall -Wextra sigcost.c -o sigcost */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <time.h>
static volatile long count = 0;
static void h(int s){ (void)s; count++; }
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
int main(void){
    struct sigaction sa={0}; sa.sa_handler=h; sigaction(SIGUSR1,&sa,NULL);
    const long N=1000000; double t0=now();
    for(long i=0;i<N;i++) raise(SIGUSR1);
    double t=now()-t0;
    printf("%ld events handled in %.3f s -> %.2f us per event\n",count,t,t/N*1e6);
    return 0; }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Source code: Task09/io_calc.c

io_calc.c

```c
/* io_calc.c - Task 9 alternative: 500 MB MRI image - interrupt per word vs DMA.
 * Build: gcc -std=c11 -Wall -Wextra -o io_calc io_calc.c */
#include <stdio.h>
#include <string.h>

static const char *commas(long long v, char *out)       /* 131072000 -> "131,072,000" */
{
    char t[32]; int n = sprintf(t, "%lld", v), j = 0;
    for (int i = 0; i < n; i++) { if (i && (n - i) % 3 == 0) out[j++] = ','; out[j++] = t[i]; }
    out[j] = 0; return out;
}

int main(void)
{
    long long size = 500LL * 1024 * 1024;
    int word = 4;
    double us_per_int = 1.5;      /* rounded from repeated sigcost runs (1.37 - 1.59 us); a real ISR
     costs at least this */
    double rate = 1e9;            /* scanner / PCIe link 1 GB/s */
    char a[32], b[32];
    printf("image = %s bytes, word = %d bytes, device rate = %.0f GB/s\n", commas(size, a), word, ra
    te / 1e9);
    long long ints = size / word;
    double cpu = ints * us_per_int * 1e-6;
    printf("Interrupt-driven I/O : %13s interrupts   CPU time %8.1f s   effective rate %6.1f MB/s
    buffer: 1 word (%d B)\n",
           commas(ints, a), cpu, size / cpu / 1e6, word);
    int blk[3] = {4096, 65536, 1048576};
    for (int i = 0; i < 3; i++) {
        long long n = size / blk[i];
        printf("DMA, interrupt per %4d KB: %9s interrupts   CPU time %8.2f ms  transfer %5.2f s   bu
    ffer: %d KB\n",
               blk[i] / 1024, commas(n, b), n * us_per_int * 1e-6 * 1000, size / rate, blk[i] / 1024
    );
    }
    printf("\nratio of interrupts: %s : 1  (word vs 64 KB DMA block)\n", commas(ints / (size / 65536
    ), a));
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 9.4: Cost of one event and interrupt/CPU-time calculation

```
student@csa04: ~/Assignment1/Task09
student@csa04:~/Assignment1/Task09$ gcc -std=gnu11 -Wall -Wextra -O2 -o sigcost sigcost.c

student@csa04:~/Assignment1/Task09$ ./sigcost
1000000 events handled in 1.347 s -> 1.35 us per event
student@csa04:~/Assignment1/Task09$ gcc -std=gnu11 -Wall -Wextra -o io_calc io_calc.c

student@csa04:~/Assignment1/Task09$ ./io_calc
image = 524,288,000 bytes, word = 4 bytes, device rate = 1 GB/s
Interrupt-driven I/O :   131,072,000 interrupts   CPU time    196.6 s   effective rate    2.7 MB/s   buffer: 1 word (4
 B)
DMA, interrupt per    4 KB:   128,000 interrupts   CPU time   192.00 ms  transfer  0.52 s   buffer: 4 KB
DMA, interrupt per   64 KB:     8,000 interrupts   CPU time    12.00 ms  transfer  0.52 s   buffer: 64 KB
DMA, interrupt per 1024 KB:       500 interrupts   CPU time     0.75 ms  transfer  0.52 s   buffer: 1024 KB

ratio of interrupts: 16,384 : 1  (word vs 64 KB DMA block)
student@csa04:~/Assignment1/Task09$
```

Q.No 10 – Measure and Compare

Source code: Task10/cp_buf.c

cp_buf.c

```c
/* cp_buf.c - copy a file with a chosen buffer size:  ./cp_buf src dst bufsize   (Task 10: strace -c
     and time) */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
int main(int argc, char **argv)
{
    if (argc != 4) { fprintf(stderr, "usage: %s src dst bufsize\n", argv[0]); return 2; }
    size_t bs = strtoul(argv[3], NULL, 10);
    char *buf = malloc(bs);
    int in = open(argv[1], O_RDONLY), out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (in < 0 || out < 0 || !buf) { perror("open"); return 1; }
    ssize_t n;
    while ((n = read(in, buf, bs)) > 0)
        if (write(out, buf, (size_t)n) != n) { perror("write"); return 1; }
    close(in); close(out); free(buf);
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 10.1: time output of cp_buf

```
student@csa04: ~/Assignment1/Task10
student@csa04:~/Assignment1/Task10$ gcc -O2 -std=gnu11 -Wall -Wextra -o cp_buf cp_buf.c

student@csa04:~/Assignment1/Task10$ ls -l data4m.dat
-rw-r--r-- 1 root root 4194304 Nov  5 05:08 data4m.dat
student@csa04:~/Assignment1/Task10$ time ./cp_buf data4m.dat out.dat 1

real  0m2.644s
user  0m0.358s
sys  0m2.228s
student@csa04:~/Assignment1/Task10$ time ./cp_buf data4m.dat out.dat 512

real  0m0.011s
user  0m0.004s
sys  0m0.004s
student@csa04:~/Assignment1/Task10$ time ./cp_buf data4m.dat out.dat 4096

real  0m0.006s
user  0m0.000s
sys  0m0.004s
student@csa04:~/Assignment1/Task10$ time ./cp_buf data4m.dat out.dat 65536

real  0m0.006s
user  0m0.000s
sys  0m0.004s
student@csa04:~/Assignment1/Task10$
```

Figure 10.2: System-call summary

```
student@csa04: ~/Assignment1/Task10
student@csa04:~/Assignment1/Task10$ ../tools/minitrace -c ./cp_buf data4m.dat out.dat 4096 2>&1 | head -5
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 45.54    0.005014           5      1024           write
 36.88    0.004060           4      1026           read
 12.81    0.001411         353         4           openat
student@csa04:~/Assignment1/Task10$ ../tools/minitrace -c ./cp_buf data4m.dat out.dat 65536 2>&1 | head -5
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 46.55    0.002350         588         4           openat
 29.08    0.001468          23        64           write
 18.04    0.000911          14        66           read
student@csa04:~/Assignment1/Task10$
```

Source code: Task10/fork_vs_thread.c (part 1 of 2)

fork_vs_thread.c (1/2)

```c
/* fork_vs_thread.c - create 1,000 processes (fork) and 1,000 threads (pthread_create); measure time and memory.
 * Build: gcc -O2 -std=gnu11 -Wall -Wextra -pthread fork_vs_thread.c -o fork_vs_thread */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
#include <sys/resource.h>
#define N 1000
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
static void *worker(void *a){ (void)a; return NULL; }
static void *sleeper(void *a){ (void)a; usleep(300000); return NULL; }
static long status_kb(const char *key){ FILE *f=fopen("/proc/self/status","r"); char l[256]; long v=0;
    while(f && fgets(l,sizeof l,f)) if(!strncmp(l,key,strlen(key))){ sscanf(l+strlen(key),"%ld",&v); break; }
    if(f){ fclose(f); }
    return v; }
int main(void){
    size_t mb=64; char *big=malloc(mb<<20); memset(big,7,mb<<20);        /* a 64 MB parent, as a database process would be */
    struct rusage a,b; double t0,t1;
    /* 1. create + finish 1000 processes, one after another */
    getrusage(RUSAGE_SELF,&a); getrusage(RUSAGE_CHILDREN,&b); t0=now();
    for(int i=0;i<N;i++){ pid_t p=fork(); if(p==0) _exit(0); waitpid(p,NULL,0); }
    t1=now(); double tp=t1-t0;
    /* 2. create + join 1000 threads, one after another */
    t0=now();
    for(int i=0;i<N;i++){ pthread_t th; pthread_create(&th,NULL,worker,NULL); pthread_join(th,NULL); }
    t1=now(); double tt=t1-t0;
    printf("Parent heap touched : %zu MB\n",mb);
    printf("1000 x fork()+wait  : %.4f s  (%.1f us each)\n",tp,tp/N*1e6);
    printf("1000 x thread+join  : %.4f s  (%.1f us each)\n",tt,tt/N*1e6);
    printf("speed ratio process/thread = %.1f x\n",tp/tt);
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

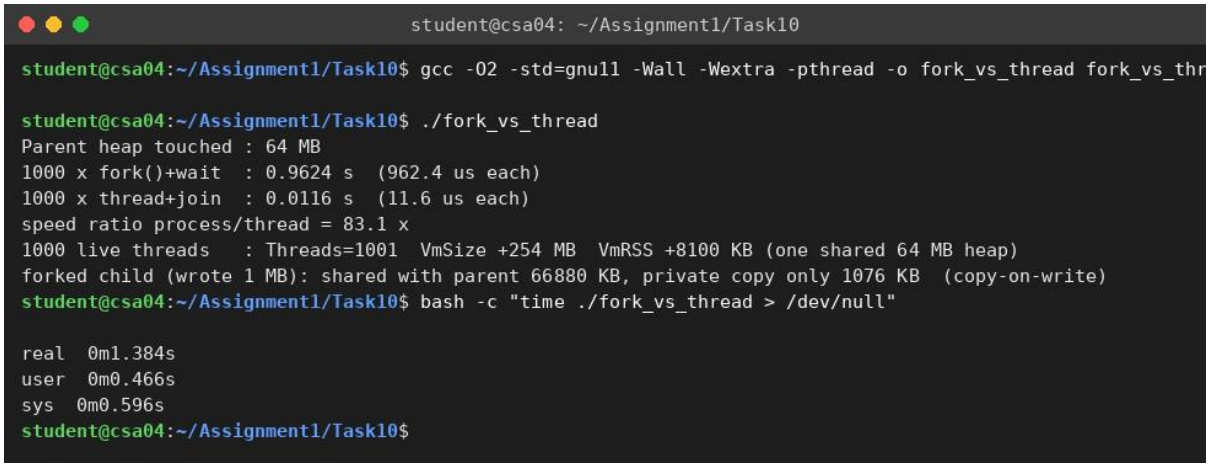
Source code: Task10/fork_vs_thread.c (part 2 of 2)

fork_vs_thread.c (2/2)

```c
    /* 3. memory: 1000 threads alive together */
    long vm0=status_kb("VmSize:"), rss0=status_kb("VmRSS:");
    pthread_t *th=malloc(N*sizeof *th); pthread_attr_t at; pthread_attr_init(&at); pthread_attr_setstacksize(&at,256*1024);
    for(int i=0;i<N;i++) pthread_create(&th[i],&at,sleeper,NULL);
    long vm1=status_kb("VmSize:"), rss1=status_kb("VmRSS:"), nth=status_kb("Threads:");
    for(int i=0;i<N;i++) pthread_join(th[i],NULL);
    printf("1000 live threads   : Threads=%ld  VmSize +%ld MB  VmRSS +%ld KB (one shared 64 MB heap)\n",nth,(vm1-vm0)/1024,rss1-rss0);
    fflush(stdout);                                  /* empty the buffer before fork() (see Task 7) */
    /* 4. memory: private pages of ONE forked child after it writes 1 MB (copy-on-write) */
    pid_t p=fork();
    if(p==0){ for(size_t i=0;i<(1u<<20);i+=4096) big[i]=9;
        FILE *f=fopen("/proc/self/smaps_rollup","r"); char l[256]; long shared=0,priv=0;
        while(f&&fgets(l,sizeof l,f)){ long v; if(sscanf(l,"Shared_Clean: %ld",&v)==1) shared+=v; if(sscanf(l,"Shared_Dirty: %ld",&v)==1) shared+=v;
            if(sscanf(l,"Private_Dirty: %ld",&v)==1) priv+=v; }
        printf("forked child (wrote 1 MB): shared with parent %ld KB, private copy only %ld KB  (copy-on-write)\n",shared,priv); fflush(stdout); _exit(0); }
    waitpid(p,NULL,0); (void)rss0;(void)rss1; free(big); return 0; }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 10.3: Creation time and memory, processes vs threads



Q.No 11 – Recommend and Defend the Operating-System Plan

Source code: Task11/vm_calc.c

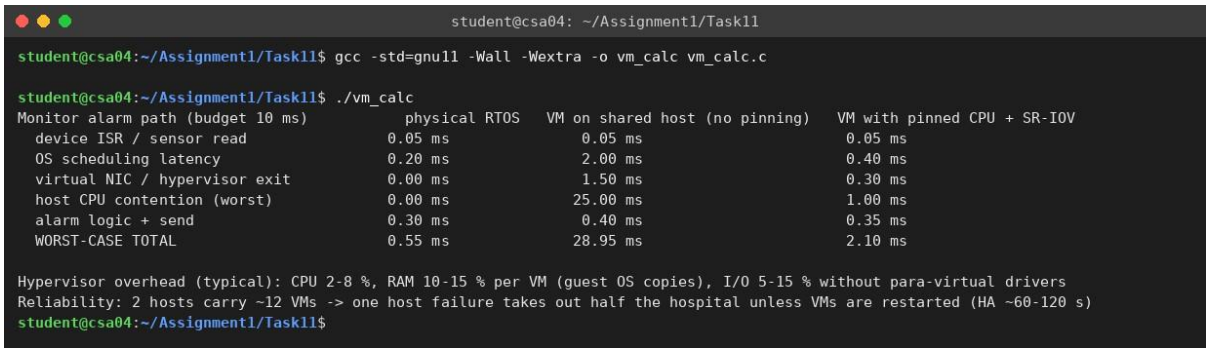
vm_calc.c

```c
/* vm_calc.c - Task 11 alternative: everything in VMs on two large servers (assumed typical overheads).
 * Build: gcc -std=c11 -Wall -Wextra -o vm_calc vm_calc.c */
#include <stdio.h>

int main(void)
{
    const char *n[5] = {"device ISR / sensor read", "OS scheduling latency", "virtual NIC / hypervisor exit",
                        "host CPU contention (worst)", "alarm logic + send"};
    double v[5][3] = {{0.05, 0.05, 0.05}, {0.20, 2.00, 0.40}, {0.00, 1.50, 0.30}, {0.00, 25.0, 1.00}, {0.30, 0.40, 0.35}};
    double t[3] = {0, 0, 0};
    printf("Monitor alarm path (budget 10 ms)           physical RTOS   VM on shared host (no pinning)   VM with pinned CPU + SR-IOV\n");
    for (int i = 0; i < 5; i++) {
        printf("  %-34s%10.2f ms %18.2f ms %26.2f ms\n", n[i], v[i][0], v[i][1], v[i][2]);
        for (int k = 0; k < 3; k++) t[k] += v[i][k];
    }
    printf("  %-34s%10.2f ms %18.2f ms %26.2f ms\n", "WORST-CASE TOTAL", t[0], t[1], t[2]);
    printf("\nHypervisor overhead (typical): CPU 2-8 %%, RAM 10-15 %% per VM (guest OS copies), I/O 5-15 %% without para-virtual drivers\n");
    printf("Reliability: 2 hosts carry ~12 VMs -> one host failure takes out half the hospital unless VMs are restarted (HA ~60-120 s)\n");
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 11.2: Output of vm_calc.c



Q.No 12 – Day-4 Design Challenge

Source code: Task12/home_budget.c

home_budget.c

```c
/* home_budget.c - Task 12: home monitoring + cloud records: load, latency and availability budget (planning estimates).
 * Build: gcc -std=c11 -Wall -Wextra -o home_budget home_budget.c */
#include <stdio.h>

int main(void)
{
    int homes = 3000, hz = 1, bytes_per = 200;
    printf("Home devices: %d, %d reading/s, %d B each -> %.0f KB/s  = %.1f GB/day\n", homes, hz, bytes_per,
           homes * hz * bytes_per / 1e3, homes * hz * bytes_per * 86400.0 / 1e9);
    printf("\nAlarm path for a home device                      ms (typical / worst)\n");
    printf("  A. cloud decides (device -> internet -> cloud)   RTT 40 / 800   + queue 20 / 500  = %d / %d ms   -> cannot meet 10 ms\n", 40 + 20, 800 + 500);
    printf("  B. edge decides on device (embedded RTOS)        sensor 0.05 + task 0.5 = 0.6 / 2 ms (alarm sounds locally), then notify cloud in 1-2 s\n");
    printf("\nAvailability of the records service\n");
    const char *nm[2] = {"single cloud region", "2 regions active/standby"};
    double a[2] = {0.999, 1 - (1 - 0.999) * (1 - 0.999)};
    for (int i = 0; i < 2; i++)
        printf("  %-26s %8.4f %%   downtime %8.1f min/year\n", nm[i], a[i] * 100, (1 - a[i]) * 8760 * 60);
    printf("\nPatch/update exposure: 500 hospital devices + 3000 homes -> over-the-air signed updates, staggered 5 %% / day\n");
    return 0;
}
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

Figure 12.1: Output of home_budget.c

student@csa04: ~/Assignment1/Task12

```
student@csa04:~/Assignment1/Task12$ gcc -std=gnu11 -Wall -Wextra -o home_budget home_budget.c

student@csa04:~/Assignment1/Task12$ ./home_budget
Home devices: 3000, 1 reading/s, 200 B each -> 600 KB/s  = 51.8 GB/day

Alarm path for a home device                      ms (typical / worst)
  A. cloud decides (device -> internet -> cloud)   RTT 40 / 800   + queue 20 / 500  = 60 / 1300 ms   -> cannot meet 10 ms
  B. edge decides on device (embedded RTOS)        sensor 0.05 + task 0.5 = 0.6 / 2 ms (alarm sounds locally), then notify cloud in 1-2 s

Availability of the records service
  single cloud region         99.9000 %   downtime    525.6 min/year
  2 regions active/standby    99.9999 %   downtime      0.5 min/year

Patch/update exposure: 500 hospital devices + 3000 homes -> over-the-air signed updates, staggered 5 % / day
student@csa04:~/Assignment1/Task12$
```

Appendix B – Source Code of tools/minitrace.c

minitrace.c (part 1 of 5)

minitrace.c (1/5)

```c
/* minitrace.c - a small strace-style system-call tracer built on ptrace(2).
 * Usage: ./minitrace [-c] program [args...]
 * Default : prints every system call as  name(args) = result   (to stderr)
 * -c      : prints a summary table (calls, errors, time) like  strace -c
 * Compile : gcc -std=gnu11 -Wall -Wextra -o minitrace minitrace.c           */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <time.h>
#include <sys/ptrace.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <sys/types.h>

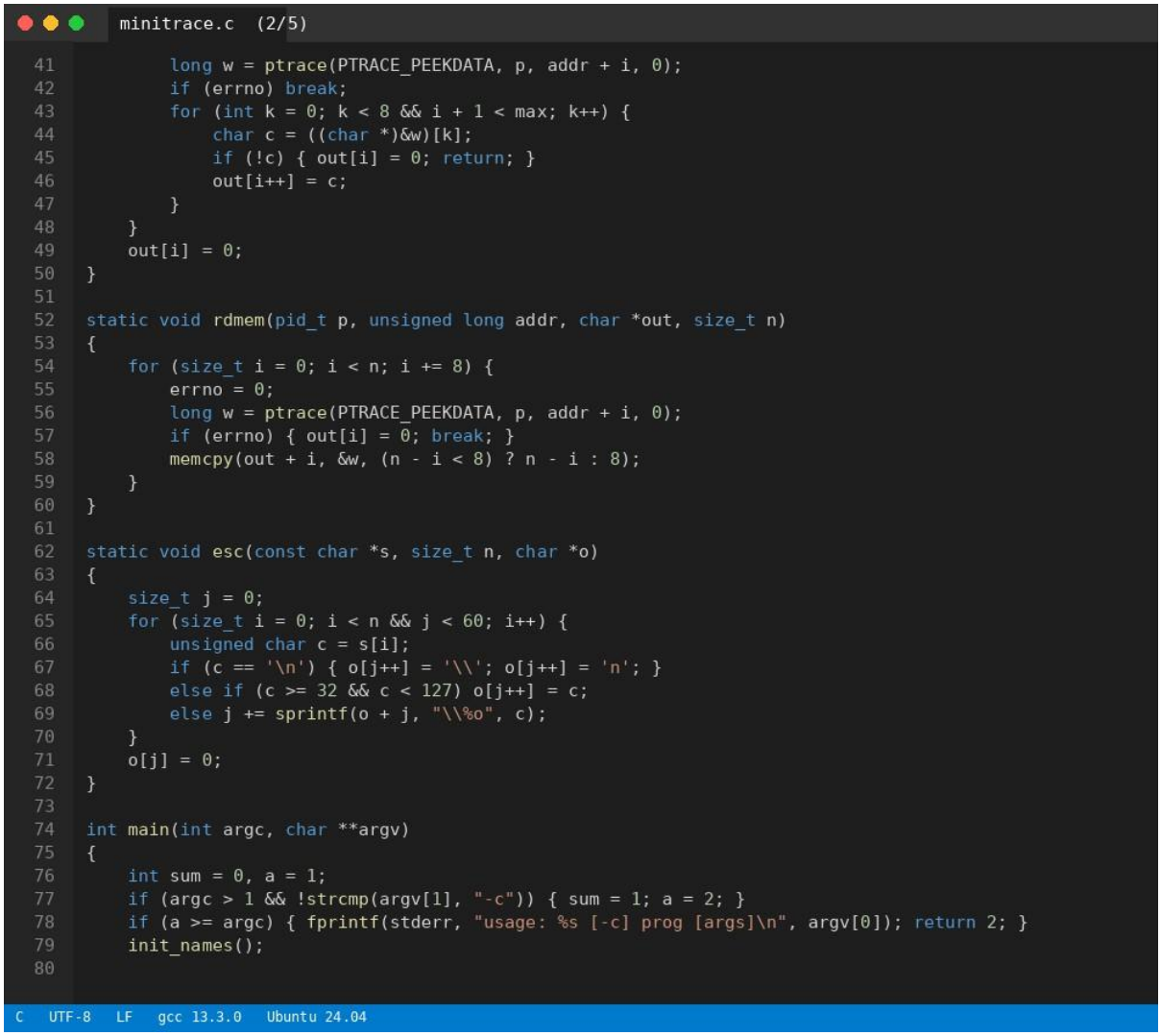
static const char *nm[512];
static void init_names(void)
{
    nm[0]="read"; nm[1]="write"; nm[2]="open"; nm[3]="close"; nm[4]="stat"; nm[5]="fstat";
    nm[7]="poll"; nm[8]="lseek"; nm[9]="mmap"; nm[10]="mprotect"; nm[11]="munmap"; nm[12]="brk";
    nm[13]="rt_sigaction"; nm[14]="rt_sigprocmask"; nm[15]="rt_sigreturn"; nm[16]="ioctl";
    nm[17]="pread64"; nm[21]="access"; nm[25]="mremap"; nm[28]="madvise"; nm[33]="dup2";
    nm[34]="pause"; nm[35]="nanosleep"; nm[37]="alarm"; nm[38]="setitimer"; nm[39]="getpid";
    nm[56]="clone"; nm[57]="fork"; nm[58]="vfork"; nm[59]="execve"; nm[60]="exit"; nm[61]="wait4";
    nm[72]="fcntl"; nm[87]="unlink"; nm[89]="readlink"; nm[158]="arch_prctl"; nm[202]="futex";
    nm[204]="sched_getaffinity"; nm[217]="getdents64"; nm[218]="set_tid_address";
    nm[228]="clock_gettime"; nm[230]="clock_nanosleep"; nm[231]="exit_group"; nm[257]="openat";
    nm[262]="newfstatat"; nm[263]="unlinkat"; nm[273]="set_robust_list"; nm[302]="prlimit64";
    nm[318]="getrandom"; nm[332]="statx"; nm[334]="rseq"; nm[435]="clone3";
}

static struct { long calls, errs; double t; } st[512];

static void rdstr(pid_t p, unsigned long addr, char *out, size_t max)
{
    size_t i = 0;
    while (i + 1 < max) {
        errno = 0;
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

minitrace.c (part 2 of 5)



minitrace.c (part 3 of 5)

minitrace.c (3/5)

```
81      pid_t c = fork();
82      if (c == 0) {
83          ptrace(PTRACE_TRACEME, 0, 0, 0);
84          execvp(argv[a], argv + a);
85          perror("execvp");
86          _exit(127);
87      }
88      int s;
89      waitpid(c, &s, 0);
90      ptrace(PTRACE_SETOPTIONS, c, 0, PTRACE_O_EXITKILL | PTRACE_O_TRACESYSGOOD);
91
92      int entering = 1;
93      long cur = -1;
94      struct user_regs_struct r;
95      struct timespec t0, t1;
96      char line[400];
97
98      while (1) {
99          ptrace(PTRACE_SYSCALL, c, 0, 0);
100         waitpid(c, &s, 0);
101         if (WIFEXITED(s) || WIFSIGNALED(s)) break;
102         if (!(WIFSTOPPED(s) && (WSTOPSIG(s) & 0x80))) continue;
103         ptrace(PTRACE_GETREGS, c, 0, &r);
104
105         if (entering) {
106             cur = r.orig_rax;
107             clock_gettime(CLOCK_MONOTONIC, &t0);
108             if (!sum) {
109                 const char *name = (cur >= 0 && cur < 512 && nm[cur]) ? nm[cur] : NULL;
110                 char d[220] = "", s1[160];
111                 switch (cur) {
112                 case 0:
113                     snprintf(d, sizeof d, "%llu, 0x%llx, %llu", r.rdi, r.rsi, r.rdx); break;
114                 case 1: {
115                     size_t n = r.rdx < 24 ? r.rdx : 24;
116                     char buf[40] = {0};
117                     rdmem(c, r.rsi, buf, n);
118                     esc(buf, n, s1);
119                     snprintf(d, sizeof d, "%llu, \"%s\"%s, %llu", r.rdi, s1, r.rdx > 24 ? "..." : ""
    , r.rdx);
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

minitrace.c (part 4 of 5)

minitrace.c (4/5)

```
120                     break; }
121                 case 257:
122                     rdstr(c, r.rsi, s1, sizeof s1);
123                     snprintf(d, sizeof d, "AT_FDCWD, \"%s\", 0x%llx", s1, r.rdx); break;
124                 case 21: case 87: case 59:
125                     rdstr(c, r.rdi, s1, sizeof s1);
126                     snprintf(d, sizeof d, "\"%s\"", s1); break;
127                 case 3: case 231: case 60:
128                     snprintf(d, sizeof d, "%llu", r.rdi); break;
129                 case 9:
130                     snprintf(d, sizeof d, "NULL, %llu, 0x%llx, 0x%llx, %lld, 0", r.rsi, r.rdx, r.r10
    , (long long)r.r8); break;
131                 default:
132                     snprintf(d, sizeof d, "..."); }
133                 if (name) snprintf(line, sizeof line, "%s(%s)", name, d);
134                 else snprintf(line, sizeof line, "syscall_%ld(%s)", cur, d);
135             }
136             entering = 0;
137         } else {
138             clock_gettime(CLOCK_MONOTONIC, &t1);
139             long ret = (long)r.rax;
140             entering = 1;
141             if (cur >= 0 && cur < 512) {
142                 st[cur].calls++;
143                 if (ret < 0 && ret > -4096) st[cur].errs++;
144                 st[cur].t += (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
145             }
146             if (!sum) {
147                 if (ret < 0 && ret > -4096)
148                     fprintf(stderr, "%-62s = -1 E%ld (%s)\n", line, -ret, strerror(-ret));
149                 else
150                     fprintf(stderr, "%-62s = %ld\n", line, ret);
151             }
152         }
153     }
154
155     if (sum) {
156         double tot = 0; long ct = 0;
157         for (int i = 0; i < 512; i++) { tot += st[i].t; ct += st[i].calls; }
158         fprintf(stderr, "%% time     seconds  usecs/call     calls    errors syscall\n");
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04

minitrace.c (part 5 of 5)

minitrace.c (5/5)

```
159         fprintf(stderr, "------ ----------- ----------- --------- --------- ----------------\n");
160         int done[512] = {0};
161         for (int k = 0; k < 512; k++) {
162             int b = -1;
163             for (int i = 0; i < 512; i++)
164                 if (!done[i] && st[i].calls && (b < 0 || st[i].t > st[b].t)) b = i;
165             if (b < 0) break;
166             done[b] = 1;
167             char nb[24], eb[16] = "";
168             if (nm[b]) snprintf(nb, sizeof nb, "%s", nm[b]); else snprintf(nb, sizeof nb, "syscall_%
    d", b);
169             if (st[b].errs) snprintf(eb, sizeof eb, "%ld", st[b].errs);
170             fprintf(stderr, "%6.2f %11.6f %11.0f %9ld %9s %s\n",
171                     tot ? 100 * st[b].t / tot : 0, st[b].t, st[b].t / st[b].calls * 1e6, st[b].calls
    , eb, nb);
172         }
173         fprintf(stderr, "------ ----------- ----------- --------- --------- ----------------\n");
174         fprintf(stderr, "100.00 %11.6f             %9ld %9s total\n", tot, ct, "");
175     }
176     return 0;
177 }
```

C UTF-8 LF gcc 13.3.0 Ubuntu 24.04